Problem Title

First Missing Positive

Difficulty Level: Hard

Problem Description (Including Scenario)

You are given an **unsorted integer array** `nums`.
Your task is to find the **smallest positive integer (≥ 1)** that is **missing** from the array.

⚠️ **Strict Requirement:**

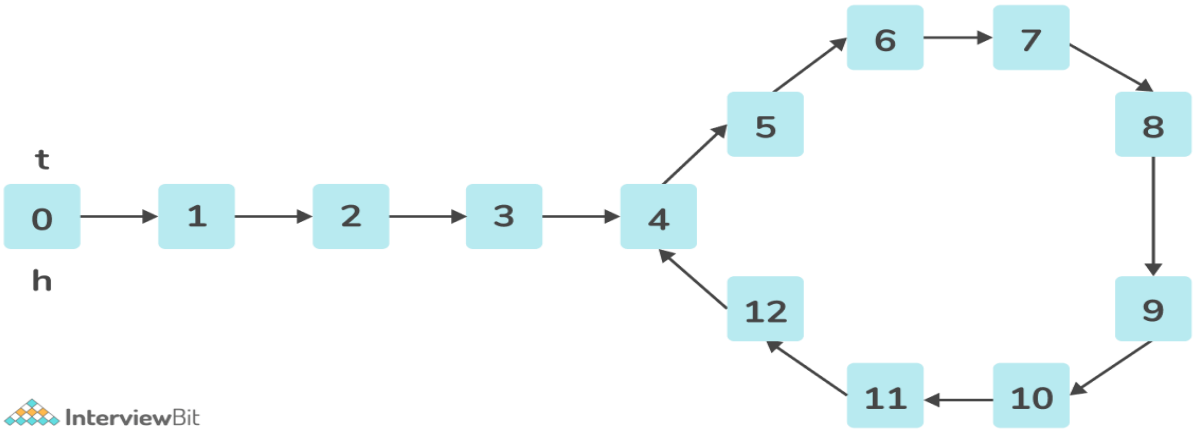
- **Time Complexity:** `O(n)`
- **Auxiliary Space:** `O(1)` (no extra data structures allowed)

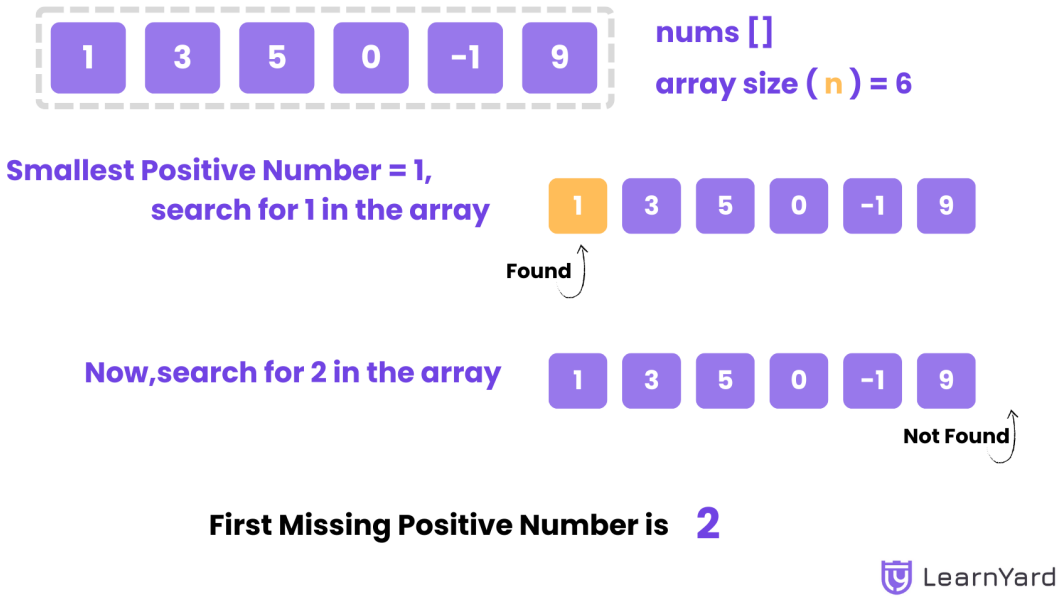
This problem is a classic test of:

- In-place array manipulation
- Index mapping logic
- Constraint-driven optimization

It is frequently asked in **top product-based company interviews**.

Visualized Example (Explanation)





Example 1

```
Input: nums = [1, 2, 0]
```

- Positive numbers present: `1, 2`
- Smallest missing positive = `3`

✅ Output:

```
3
```

Example 2

```
Input: nums = [3, 4, -1, 1]
```

Rearranged logically (in-place idea):

```
Index:  0  1  2  3
Value:  1 -1  3  4
```

- `1` is present
- `2` is missing

✅ Output:

```
2
```

Example 3

```
Input: nums = [7, 8, 9, 11, 12]
```

- No `1` present

✅ Output:

```
1
```

Input Format

- An integer array `nums` of length `n`

```
nums = [3,4,-1,1]
```

Output Format

- Return a **single integer** representing the smallest missing positive number

```
2
```

Constraints

```
1 ≤ nums.length ≤ 10^5
-2^31 ≤ nums[i] ≤ 2^31 - 1
```

Topic Covered

- Arrays
- In-place Algorithms
- Index Mapping
- Time & Space Optimization

Approaches

🔷 1. Brute Force Approach (Not Allowed)

Idea:
Check from `1` onwards whether each number exists in the array.

Time Complexity: `O(n²)`
Space Complexity: `O(1)`

❌ Too slow for large inputs

🔷 2. Sorting Approach (Not Optimal)

Idea:
Sort the array and find the first missing positive.

Time Complexity: `O(n log n)`
Space Complexity: `O(1)` / `O(n)` depending on sort

❌ Violates time constraint

---

3. Optimized Approach – Index Placement (Cyclic Sort) ✅

Core Idea:
If a number `x` is in the range `[1, n]`, it **belongs at index `x - 1`**.

Algorithm:

1. Iterate through the array
2. While `nums[i]` is in `[1, n]` and not in its correct position:
   - Swap it to index `nums[i] - 1`
3. After rearrangement, scan array:
   - First index `i` where `nums[i] != i + 1` → answer is `i + 1`
4. If all positions are correct → answer is `n + 1`

Time Complexity: `O(n)`
Space Complexity: `O(1)`

✓ **Interview-preferred solution**

Java Solution (Preferred Language)

```
class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;

        // Place each number in its correct index position
        for (int i = 0; i < n; i++) {
            while (nums[i] >= 1 && nums[i] <= n
                   && nums[nums[i] - 1] != nums[i]) {

                int correctIndex = nums[i] - 1;
                int temp = nums[i];
                nums[i] = nums[correctIndex];
                nums[correctIndex] = temp;
            }
        }

        // Find the first missing positive
        for (int i = 0; i < n; i++) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }

        return n + 1;
    }
}
```

✅ O(n) time
✅ O(1) space
✅ No extra data structures
✅ Handles all edge cases

Practice Link (Live)

- **LeetCode**
  https://leetcode.com/problems/first-missing-positive/

- **GeeksforGeeks**
  https://www.geeksforgeeks.org/find-the-smallest-positive-number-missing-from-an-unsorted-array/

Video Solution Link

- **YouTube**
  https://www.youtube.com/watch?v=8g78yfzMlao

Learning Outcome

After solving this problem, the learner will be able to:

- Design true `O(n)` algorithms
- Use arrays as hash maps
- Apply cyclic sort / index placement
- Handle hard-level interview constraints
- Write optimized, in-place Java solutions